claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_98c1f280-b9f\agent-a9123814e4fa5e3bd.jsonl`
- SHA-256: `66dd383df522e7539c2ff87d59b8c67a176af0dfa3155aaa15290a2a885cbd0a`
- Source modified: `2026-06-14T06:37:27+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_98c1f280-b9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T06:34:32.754Z)

```
You are a skeptical auditor. Here are 7 findings from a code audit of the badminton-highlight-indexer repo:
[
  {
    "component": "PERSON cue (person_detector.py producer + fusion_features.py per-window feature math, wired through FusionSegmenter and the fusion_golden/fusion_compare LOVO measurement)",
    "present": true,
    "default_off": true,
    "no_special_casing": true,
    "retained": true,
    "evidence": [
      "PRESENT (producer): backend/pipeline/detectors/person_detector.py:50 `class PersonDetector` ? torchvision detector + ByteTrack + motion windowing; :140 `extract_action_windows_from_chunk`, :258 `detections_per_frame`, :305 `detections_over_windows` (single-pass per-window detections feeding fusion).",
      "PRESENT (feature math): backend/pipeline/detectors/fusion_features.py defines all 5 features ? :97 `players_in_court`, :102 `in_court_ratio`, :109 `mean_player_motion`, :130 `two_sided_motion`, :168 `shuttle_player_colocation`, aggregated by :200 `compute_person_features`; :194 `PERSON_FEATURE_NAMES = ['players_in_court','two_sided_motion','mean_player_motion','in_court_ratio','shuttle_player_colocation']`. Pure (no torch/cv2/I/O), scale/translation-invariant by design.",
      "DEFAULT-OFF (cue flag): backend/config/models.py:96 `cue_flags: dict[str, bool] = Field(default_factory=dict)` (empty ? person OFF); models.py:94-95 comment 'Person cue is OFF unless explicitly enabled'. tests/test_config.py:48 `assert f.cue_flags == {}  # person cue OFF`.",
      "DEFAULT-OFF (segmenter): backend/config/models.py:112 `default_segmenter: str = 'yolo_hybrid'` and config.json:26 `\"default_segmenter\": \"gemini\"` ? neither is the `fusion_hybrid` segmenter. FusionSegmenter is opt-in: fusion_hybrid.py:23-26 'Registered as fusion_hybrid, NOT the default segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift? never silently.'",
      "DEFAULT-OFF (enrichment gated on flag): backend/pipeline/segmenters/fusion_hybrid.py:88 `if fcfg.get('cue_flags', {}).get('person', False):` guards `_person_features`; fusion_hybrid.py:109-112 lazily builds PersonDetector / loads torchvision ONLY when the flag is on, so the default path never touches the GPU.",
      "DEFAULT-OFF (served gate is a separate knob, also default 0): backend/config/models.py:102 `min_players: int = Field(default=0, ge=0)`; fusion_hybrid.py:148 `if person_on and min_players > 0 and st.get('players_in_court',0.0) < min_players: continue` ? only fires when the operator both enables person AND raises min_players>0. Default thresholds keep every window (fusion_hybrid.py:136-138).",
      "NO SPECIAL-CASING (features are learned columns): backend/eval/fusion_compare.py:6 'the person features are extra columns the learned LogisticRegression weights'; fusion_compare.py:57-61 builds two Variants differing ONLY by `feature_names` ? shuttle_only vs `SHUTTLE_FEATURE_NAMES + PERSON_FEATURE_NAMES`. experiment.py:234
```

```
`model.predict_proba(_feature_matrix(vc, variant.feature_names))` ? a LogisticRegression (experiment.py:49,293 `LogisticRegression().fit(X,y,variant.feature_names)`) scores them; there is NO hard if/else on any person feature in the decision path.",
      "NO SPECIAL-CASING (producer?feature?scorer chain): backend/eval/fusion_golden.py:108 `ff.compute_person_features(...)` produces the columns per candidate; :112-114 persists them as `person` dict next to `shuttle`; fusion_compare.py:47-48 loads them into `features[fn]`; the only hard gate (`min_players`) is the eval-side Gate B (experiment.py:200-210 `_gate_mask`) which is a SEPARATE default-0 knob, not the path by which the cue enters the learned decision.",
      "RETAINED (code intact post-measurement): n=6 result is non-significant ? eval_baselines/experiment_leaderboard.json shows shuttle_only F1=0.307 vs shuttle_person_fusion F1=0.286, delta f1=-0.0205, honest_delta_f1 ci_low=-0.165 ci_high=+0.162 ci_excludes_zero=false, sign_test 2W/4L p=0.6875. Despite this, person_detector.py and fusion_features.py are unchanged and fully functional.",
      "RETAINED (only a docs log, no code change): branch docs/i3-fusion-measured commit 9eb06f4 `docs(quality): log fusion P3+P4 measured ?F1` touches ONLY docs/QUALITY_ITERATIONS.md (+19 lines); `git diff --stat origin/master...HEAD` = `docs/QUALITY_ITERATIONS.md | 19 +++` and nothing else. Commit message: 'neither cue is promotable; both stay default-OFF (promote-only-on-lift)? Negative/uncertain results kept on purpose ? the honest ablation backbone.'",
      "RETAINED (re-measurable): fusion_golden.py writes replayable `<name>_golden_features.json` per video (the persisted feature substrate, fusion_golden.py:13-15) so `fusion_compare` can re-run forever without re-detecting; FusionSegmenter persists the full candidate superset regardless of gating (fusion_hybrid.py:18-20) so the harness can re-score any variant as the corpus grows."
    ],
    "concerns": [
      "Feature-quality caveat acknowledged in code/docs: the n=6 run was extracted with `court_polygon=None` (fusion_golden.py:109 passes `court_polygon=None`), so `players_in_court` and `two_sided_motion` count spectators / adjacent-court people. The commit msg notes 'Person adds noise (no court polygons -> players_in_court counts spectators)'. This means the non-significant/negative ?F1 partly reflects un-tuned features, not necessarily a useless cue ? re-measuring with court polygons (FusionConfig.court_polygon, models.py:104) is the obvious next experiment before any conclusion. Not a guardrail violation, but the negative result should not be read as 'person cue doesn't work'.",
      "The person cue's served Gate B (`min_players`, fusion_hybrid.py:148; experiment.py _gate_mask:207-209) IS a hard threshold if/else ? but it is a SEPARATE operator-set decision-gate (default 0 = off), distinct from how the feature enters the learned scorer. The 'no special-casing' criterion holds for the learned decision path (feature columns), but reviewers should note `players_in_court` does also have a hard-gate consumer; both are default-OFF so neither runs by default.",
      "`compute_person_features` is invoked from TWO call sites with slightly different framing: fusion_golden.py (eval, court_polygon=None) and fusion_hybrid.py:130 (served, court_polygon from config). Production wiring (fusion_hybrid._person_features) is explicitly only mock-tested ? fusion_hybrid.py:108 '? real-video verification is owner-side; this wiring is mock-tested.' So the served person path has not been exercised on real video; only the offline eval path produced the n=6 numbers. Re-measurability claim is sound, but the served path's correctness on real footage is unverified here (GPU-free container).",
      "n=6 is too small to conclude anything either direction ? both the audit's 'non-significant' framing and the code comments agree. The retention decision is correct precisely because the result is inconclusive, not because the cue was shown harmful."
    ]
  },
  {
    "component": "audio cue (audio_features.py + hit_detector.py, consumed by eval/fusion_audio.py)",
    "present": true,
    "default_off": true,
    "no_special_casing": true,
    "retained": true,
    "evidence": [
      "PRESENT producer: backend/pipeline/audio/hit_detector.py:136 `def detect_hits_in_video(video_path, sr=16000, **kw) -> List[HitEvent]` ? full onset chain extract_audio->load_wav->onset_envelope->_moving_stats->detect_hits (adaptive mean+k*std threshold, lines 28-133).",
      "PRESENT feature math: backend/pipeline/detectors/audio_features.py:15 `AUDIO_FEATURE_NAMES = [\"audio_hit_count\", \"audio_hit_rate\", \"audio_hit_strength_mean\"]`;
```

:18 `def compute_audio_features(hits, window_start, window_end) -> Dict[str,float]` returns a numeric dict (0.0 on empty window, :28). Unit-tested in tests/test_audio_features.py:16-33.",
      "DEFAULT_OFF (served default): backend/config/models.py:112 `default_segmenter: str = \"yolo_hybrid\"` and config.json:26 `\"default_segmenter\": \"gemini\"` ? never 'fusion'. Selection is `req.segmenter or default_segmenter` (backend/main.py:712, docs/CODE_MAP.md:31). Audio is unreachable from any served segmenter.",
      "DEFAULT_OFF (audio is eval-only): the ONLY importers of detect_hits_in_video/compute_audio_features are backend/eval/fusion_audio.py:24-25 and backend/eval/audio/e1_probe.py:18. `git grep` for these imports excluding backend/eval and tests returns EMPTY ? zero production importers.",
      "DEFAULT_OFF (not even in the opt-in fusion segmenter): backend/pipeline/segmenters/fusion_hybrid.py `_enrich_candidate_stats` (:76-90) persists only `net_crossings` (:85) and, behind `cue_flags['person']` (:88), person features ? it NEVER calls compute_audio_features or detect_hits_in_video. There is no `cue_flags['audio']` in FusionConfig (models.py:78-108).",
      "NO_SPECIAL_CASING: audio enters the decision purely as feature columns the learned LogisticRegression weights. backend/eval/fusion_audio.py:84 builds the variant with `feature_names = SHUTTLE + PERSON + af.AUDIO_FEATURE_NAMES`; experiment.predict (experiment.py:227-238) does `model.predict_proba(_feature_matrix(vc, variant.feature_names))` then thresholds the probability. No `if audio_hit_count > X` branch exists; _gate_mask (experiment.py:200-210) only references net_crossings/players_in_court, never audio.",
      "NO_SPECIAL_CASING (per-window math is pure numeric): audio_features.py:30-33 returns {audio_hit_count: float(n), audio_hit_rate: n/dur, audio_hit_strength_mean: mean(strength)} ? values, never a hard-coded keep/drop decision.",
      "RETAINED (no deletion): `git log --all --diff-filter=D` over both files + fusion_audio.py returns EMPTY (no commit ever removed them). Both still on master; added in #122 dc19bf2 'fusion P4 ? audio cue + incremental-lift measurement (default-OFF)'.",
      "RETAINED (kept despite non-significant n=6): docs/QUALITY_ITERATIONS.md:52 records `+ person + audio (P4 incr.) | 0.366 | +0.079 (+27.7%) | [-0.054, +0.203] | 4W/2L (p=0.688) | CI clears 0: No`; :54-60 conclude 'both stay default-OFF' and ':59-60 `Negative/uncertain results are kept on purpose ? this is the honest ablation backbone`. Re-measurable as N grows via `fusion_audio.py --augment --compare` (the producer + feature columns persist into *_golden_features.json)."
    ],
    "concerns": [
      "The real-audio producer path (detect_hits_in_video: ffmpeg extraction + numpy onset detection) has NO offline test ? this dev container lacks ffmpeg/numpy/cv2, so only the pure compute_audio_features math and the synthetic LOVO driver are covered (tests/test_audio_features.py). The actual hit-detection quality on real video is owner/GPU-side-verified only; the n=6 ?F1 came from real runs but is not reproducible in CI.",
      "Audio is NOT yet wired to ANY served segmenter, even the opt-in fusion_hybrid. Unlike Gate A (which already has a served `min_crossings` knob and a persisted net_crossings stat) and the person cue (which has `cue_flags['person']` + a `_person_features` enrichment seam), there is NO `cue_flags['audio']`, no audio enrichment seam in fusion_hybrid._enrich_candidate_stats, and audio features are never persisted into candidate_segments.stats. So even if audio measured promotable, it would require new production wiring ? it currently exists ONLY in the offline eval harness. This is correct/honest for a default-OFF cue but worth flagging: 'present + re-measurable' is true only in eval, not as a flag-flippable served path.",
      "The +0.079 audio lift is measured as an INCREMENTAL gain ON TOP of the person cue (shuttle+person vs shuttle+person+audio, fusion_audio.py:79-86), and person itself was a -0.021 regression. QUALITY_ITERATIONS.md:58 itself notes audio should next be measured on shuttle-only directly 'without the person handicap' ? so the headline number is confounded by the person baseline and is not yet a clean audio-vs-shuttle measurement."
    ]
  },
  {
    "component": "COURT BOUNDS ? court-polygon person-occupancy signal (fusion_features.point_in_polygon / person_in_court / in_court_counts / players_in_court / in_court_ratio, wired via FusionSegmenter, gated by FusionConfig.court_polygon)",
    "present": true,
    "default_off": true,
    "no_special_casing": true,
    "retained": true,
    "evidence": [
      "MATH PRESENT: backend/pipeline/detectors/fusion_features.py:30-48 `point_in_polygon`

(ray-casting, <3 verts => False); :51-57 `_foot_norm` (bottom-centre foot point = court-membership anchor); :67-76 `person_in_court` ? 'if court_polygon is None: return True' (count-everyone) else foot-point-in-polygon; :79-86 `in_court_counts` per-frame in-court count; :97-99 `players_in_court` median; :102-106 `in_court_ratio` fraction of frames >= min_players. Pure, no torch/cv2.",

"CATALYST ACTIVATION: backend/pipeline/detectors/fusion_features.py:200-215 `compute_person_features(..., court_polygon=None, ...)` threads the polygon into `in_court_counts` (:207) and `two_sided_motion` (:210). With None it degrades to 'every detection counts'; with a polygon every count/ratio/two-sided value changes ? proven by tests/test_fusion_features.py:54-67 (`in_court_counts(...,LEFT_HALF)==[1,1,0,2]` vs `(...,None)==[2,1,0,2]`; players_in_court 1.0; in_court_ratio 0.25), :38-49 (person_in_court LEFT_HALF True/False vs None always True), :94-101 (two_sided_motion 1.0 with None vs 0.0 with LEFT_HALF). So the math activates the moment a polygon is supplied.",

"PRODUCTION WIRING activates on config: backend/pipeline/segmenters/fusion_hybrid.py:126-131 ? `poly = fcfg.get('court_polygon')`; `polygon = [(float(p[0]),float(p[1])) for p in poly] if poly else None`; passed into `ff.compute_person_features(..., court_polygon=polygon, net=net)`. So once `indexing.fusion.court_polygon` is set (e.g. when Roora labels arrive) the masked path engages with no code change ? the catalyst-later mechanism.",

"DEFAULT-OFF (config): backend/config/models.py:104 `court_polygon: Optional[list[list[float]]] = None`; :96 `cue_flags: dict[str,bool] = Field(default_factory=dict)` (person cue absent => OFF); :99-102 `min_crossings`/`min_players` default 0 (=OFF). FusionConfig docstring :85-86 confirms None => Gate B counts every detection (player-count-only).",

"DEFAULT-OFF (served path): backend/config/models.py:112 `default_segmenter: str = 'yolo_hybrid'` and config.json:26 ships `'gemini'` ? neither is `fusion_hybrid`, so the served path never instantiates FusionSegmenter by default. fusion_hybrid.py:88 only computes person features `if fcfg.get('cue_flags',{}).get('person',False)` (default False) ? so the court-polygon code never even runs in the default path. fusion_hybrid.py:22-26 docstring: persists but gates nothing by default; 'Registered as fusion_hybrid, NOT the default segmenter ? it is opt-in.'",

"DEFAULT-OFF (eval): backend/eval/fusion_golden.py:108-109 calls `ff.compute_person_features(..., court_polygon=None, ...)` ? explicitly None today (the 'currently None' the prompt notes); the degrade-to-count-everyone path, re-runnable with a polygon later.",

"NO SPECIAL-CASING: the court-derived person features enter the DECISION only as learned-scorer columns. backend/eval/fusion_compare.py:47-48 loads `players_in_court`/`in_court_ratio`/etc. into the feature dict; :59-61 the fusion Variant differs from shuttle_only ONLY by `feature_names = SHUTTLE + PERSON`; the LogisticRegression (distill_local.py:116/135 `LogisticRegression().fit(X,y,FEATURE_NAMES)`) weights them. fusion_compare.py docstring :5-7: 'the person features are extra columns the learned LogisticRegression weights ... not a hunch.' The ONLY if/else on a court feature is the OPTIONAL served Gate B (fusion_hybrid.py:148 `if person_on and min_players>0 and st['players_in_court']<min_players`), which is a config-gated threshold defaulting OFF (min_players=0), NOT a hard-coded decision branch.",

"RETAINED despite non-significant n=6: git log shows only feature-ADD commits on these files (cbf2236 P2, 39e961b P3, d7b1f51 honest ?F1) ? no deletion/disable commit after the result. docs/QUALITY_ITERATIONS.md:51 records +person ?F1 ?0.021, CI [?0.165,+0.162], 2W/4L p=0.688, 'CI clears 0: No'; :54-55 'both stay default-OFF (promote-only-on-lift)'; :55 explicitly attributes the miss to 'no court polygons ? players_in_court counts spectators'; :59-60 'Negative/uncertain results are kept on purpose ? this is the honest ablation backbone'. Producer (fusion_golden.py) + feature math (fusion_features.py) + scorer columns (fusion_compare.py) all still exist and are re-measurable when the corpus grows. docs/NEXT_STEPS.md:71-72 keeps 'Court mask = optional fusion.court_polygon' as a live capability."

],

"concerns": [

"No real-video activation of the polygon path has been run: fusion_hybrid.py:108 flags person-feature wiring as 'real-video verification is owner-side; this wiring is mock-tested', and CLAUDE.md notes the dev container has no GPU/torch/cv2. The polygon MATH is unit-tested with LEFT_HALF, but the end-to-end masked person-detection pass on actual video (and thus the catalyst's real-world lift) is unverified ? by design, owner-gated.",

"No court_polygon is committed anywhere yet (config.json has no fusion block; FusionConfig default None; fusion_golden passes None literally). The catalyst is fully plumbed but dormant ? it only fires after someone supplies the polygon (the prompt's 'once Roora labels arrive').

```
There is no producer that auto-derives a polygon from court detection today; it must be
hand/label-supplied via config.",
      "Coordinate-space coupling risk for the catalyst: polygons must be normalised 0-1 and
`two_sided_motion`/foot-points use per-axis normalisation (x/width, y/height) ? a future Roora-
supplied polygon in pixel space or a different convention would silently mis-mask
(point_in_polygon returns plausible-but-wrong booleans, not an error). No validation that
court_polygon vertices are in [0,1] exists in FusionConfig (only `Optional[list[list[float]]]`);
a malformed polygon degrades silently rather than failing loudly.",
      "Minor: the n=6 person result is reported as a COMBINED person-feature-set ?F1; the court-
occupancy sub-signal's individual contribution is not separately measured, so 'court bounds
specifically helps' remains untested even setting aside the None-polygon handicap. Re-measuring
per-feature once polygons exist would be the honest next step."
    ]
  },
  {
    "component": "FusionSegmenter + config defaults
(backend/pipeline/segmenters/fusion_hybrid.py, trajectory_hybrid.py, backend/config/models.py)",
    "present": true,
    "default_off": true,
    "no_special_casing": true,
    "retained": true,
    "evidence": [
      "PRESENT: backend/pipeline/segmenters/fusion_hybrid.py:39 `class
FusionSegmenter(TrajectoryHybridSegmenter):` with `family = \"fusion\"` (:42) and `name ->
\"fusion_hybrid\"` (:56); registered at :154 `segmenter_registry.register(FusionSegmenter)`.
Producers exist and are git-tracked: backend/pipeline/detectors/fusion_features.py
(PERSON_FEATURE_NAMES def at :194), rally_gate.py (Gate A net-crossing).",
      "PRESENT (config): backend/config/models.py:78 `class FusionConfig` wired into
IndexingConfig:142 `fusion: FusionConfig = Field(default_factory=FusionConfig)`.",
      "DEFAULT_OFF (segmenter): model default is NOT fusion ? models.py:112 `default_segmenter:
str = \"yolo_hybrid\"`; shipped config.json:26 `\"default_segmenter\": \"gemini\"`. Neither is
fusion. The only literal `\"fusion_hybrid\"` in backend/ is the segmenter's own name property
(fusion_hybrid.py:56) ? never assigned as a default. Module docstring fusion_hybrid.py:24-26
states it is 'Registered as fusion_hybrid, NOT the default segmenter ? it is opt-in.'",
      "DEFAULT_OFF (cues): FusionConfig.cue_flags defaults empty -> person OFF. models.py:96
`cue_flags: dict[str, bool] = Field(default_factory=dict)`; gate read at fusion_hybrid.py:88 `if
fcfg.get(\"cue_flags\", {}).get(\"person\", False):` (default False ? no person detector / no
torch model loaded, :110-112 lazy import only when ON).",
      "DEFAULT_OFF (served gates no-op): models.py:99 `min_crossings: int = Field(default=0,
ge=0)` and :102 `min_players: int = Field(default=0, ge=0)`. fusion_hybrid.py:146-150 only drops
windows when `min_crossings > 0` / `min_players > 0`; at defaults `_select_for_handoff` keeps
every window ? docstring :136-138 'Default thresholds (0) keep every window ? identical to the
substrate.' Base class confirms identity: trajectory_hybrid.py:104 `return
list(range(len(windows)))`.",
      "NO_SPECIAL_CASING: the cue enters the DECISION as a feature COLUMN a learned
LogisticRegression weights, not a hard-coded decision branch. backend/eval/fusion_compare.py:5-7
'the two variants differ ONLY by feature_names ... the person features are extra columns the
learned LogisticRegression weights'. fusion_compare.py:57-61 builds variants whose only
difference is `feature_names=SHUTTLE_FEATURE_NAMES` vs `+ PERSON_FEATURE_NAMES`.
experiment.py:227-238 `predict()` runs `model.predict_proba(_feature_matrix(vc,
variant.feature_names))` then thresholds the learned probability; experiment.py:281-293 `_train`
fits `LogisticRegression().fit(X, y, variant.feature_names)`. `net_crossings` is a learned
shuttle feature: distill_local.py:40 `FEATURE_NAMES =
[\"duration\",\"peak_velocity\",\"mean_velocity\",\"active_ratio\",\"net_crossings\"]`, re-used
as SHUTTLE_FEATURE_NAMES (fusion_golden.py:33).",
      "RETAINED: nothing was deleted/disabled despite a non-significant n=6 result.
eval_baselines/experiment_leaderboard.json (n=6) records shuttle_only vs shuttle_person_fusion
`delta.f1 = -0.0205`, `honest_delta_f1.ci_excludes_zero: false`, sign_test 2W/4L p=0.6875 ? a
clear non-significant/negative lift. Yet the producer + feature math remain: fusion_hybrid.py:85
still computes `stats[\"net_crossings\"]`, :88-89 still computes person features,
fusion_features.py/rally_gate.py/fusion_golden.py/fusion_compare.py are all still git-tracked
and importable. Re-measurable when the corpus grows (load_videos just re-reads
*_golden_features.json).",
      "PERSISTENCE INVARIANT (supports retained/re-measurable): even when served gates ARE
raised, persisted candidates stay the full superset ? fusion_hybrid.py:20 and :136-138
```

'Persisted candidates are untouched (full superset)'; trajectory_hybrid.py:208-217 persists every window's stats unconditionally; only `_select_for_handoff` indices (:225) are gated for the served handoff."
    ],
    "concerns": [
      "The served Gate A/B path in fusion_hybrid.py:_select_for_handoff IS a hard if/else threshold branch (`min_crossings > 0 ... < min_crossings`, :146-150), NOT a learned scorer. This is fine because (a) it is default-OFF (thresholds 0) so it never fires on the served path, and (b) the no_special_casing criterion is about how the cue enters the LEARNED DECISION (the eval harness uses feature_names columns, which it does). But note the served production segmenter, if ever switched to fusion_hybrid with thresholds raised, would use hard-coded if/else gating ? not the LogisticRegression. The learned-scorer treatment lives only in the offline eval harness (backend/eval/experiment.py + fusion_compare.py), not in the served fusion_hybrid path. Promotion-to-default requires owner sign-off per docstring :24-26.",
      "Two distinct gate semantics share names: experiment.py:_gate_mask (:200-210) ALSO implements min_crossings/min_players as hard thresholds on the eval side, independent of the learned classifier ? so even in eval those two knobs are hard gates layered before the learned filter. The learned-column treatment applies to feature_names (incl. net_crossings/person features), while min_crossings/min_players are deliberately a separate declarative hard gate. This is intentional (Variant docstring experiment.py:90-96) but means 'net_crossings' can be consumed BOTH as a learned column and as a hard gate depending on which knob a variant sets.",
      "config.json ships default_segmenter='gemini' while the model default is 'yolo_hybrid' ? a known divergence (CODE_MAP.md:105). Neither is fusion, so default-OFF holds, but the served default is the cloud Gemini path, not yolo_hybrid; the fusion-vs-substrate equivalence claim is about wasb_hybrid, a third path. No impact on this component's default-OFF status.",
      "Real-video verification of the person path is owner-side only (fusion_hybrid.py:108 '? real-video verification is owner-side; this wiring is mock-tested'). The dev container lacks torch/cv2, so the person-cue producer is exercised only via mocks here ? re-measurability of the person columns at corpus growth depends on owner-side GPU runs, but the math/code is fully retained.",
      "A second leaderboard row (shuttle_person vs shuttle_person_audio) shows a positive +0.079 ?F1 for an AUDIO cue ? out of scope for this component but worth noting the audio variant is a separate eval driver (fusion_audio.py); it does not change the person/net-crossing default-OFF posture audited here."
    ]
  },
  {
    "component": "SCORING seam / no-special-casing (backend/eval/experiment.py ? Variant.feature_names, compare(), the learned LogisticRegression scorer)",
    "present": true,
    "default_off": true,
    "no_special_casing": true,
    "retained": true,
    "evidence": [
      "backend/eval/experiment.py:113-114 ? Variant carries `classifier_model: Optional[str] = None` and `feature_names: Optional[List[str]] = None`; a cue enters the learned decision purely by appearing in this list. Adding/removing a cue is a feature-LIST change, not a code edit.",
      "backend/eval/experiment.py:227-238 (predict) ? the ONLY learned-decision path: `if variant.feature_names:` -> build `_feature_matrix(vc, variant.feature_names)` -> `model.predict_proba(...)` -> threshold the probability. The cue's name is data passed to the scorer; there is NO per-cue `if net_crossings.../if players...` branch in this path. `_feature_matrix` (195-197) and `_feature_column` (180-192) are name-driven and cue-agnostic (a missing column defaults to 0.0, line 186-188).",
      "backend/eval/classifier.py:40-73 (fit) + :75-79 (predict_proba) ? a generic numpy LogisticRegression over a feature matrix; it learns one WEIGHT per column (`self.w`), standardizes, sigmoids. The scorer has zero cue-specific logic ? every feature is just a column index. `feature_names` is stored as metadata only (line 48, 94).",
      "backend/eval/distill_local.py:40 ? `FEATURE_NAMES = [\"duration\", \"peak_velocity\", \"mean_velocity\", \"active_ratio\", \"net_crossings\"]` fed to `LogisticRegression().fit(X, y, FEATURE_NAMES)` (:116). PROOF the shuttle cue (`net_crossings`) already rides the learned scorer as a weighted COLUMN, not an if/else ? exactly the no-special-casing pattern. docs/MULTI_SIGNAL_FUSION_PLAN.md:128 confirms: '`net_crossings` is already a feature'.",
      "Default-OFF in production: backend/config/models.py:112 `default_segmenter: str = \"yolo_hybrid\"` and shipped config.json:26 `\"default_segmenter\": \"gemini\"` ? neither is the fusion/learned path. FusionConfig (:96/:99/:102) `cue_flags={}`, `min_crossings=0`,

```
`min_players=0` all default-OFF. Grep for any non-eval import of
`experiment`/`LogisticRegression` across backend/ (excluding eval/) returned NO matches ? the
learned scorer never runs in the served pipeline by default.",
      "no-regression invariant test: tests/test_experiment.py:78-83
`test_disabled_cues_byte_identical_to_control` asserts a variant carrying only disabled cues
`predict(...) == predict(CONTROL, ...)`. tests/test_experiment.py:129-133 shows the learned path
is a recipe (`Variant(feature_names=[\"x\"])`) recovering F1 via LOVO ? a pure feature-list
config, no code branch.",
      "Retained after non-significant n=6 ?F1: memory current-state.md:12-17 records measured
honest LOVO (n=6): +person P3 ?F1=-0.021 (CI spans 0), +person+audio P4 ?F1=+0.079 (CI spans 0),
'neither promotable'. Despite that, the producers + feature math are all still present and re-
measurable: backend/pipeline/detectors/fusion_features.py (person feature math),
audio_features.py, backend/eval/fusion_golden.py/fusion_compare.py/fusion_audio.py, and the
experiment seam itself ? nothing deleted/disabled. fusion_hybrid.py:22-26 docstring: persisted
candidates remain the full superset for offline re-scoring; promotion 'happens only on measured
LOVO lift... never silently.'"
    ],
    "concerns": [
      "NOT strictly 'cues enter ONLY as feature columns'. The harness retains a SECOND,
deliberate decision path: hard-threshold Gate A/B. In the offline Variant, `_gate_mask`
(experiment.py:200-210) does `if variant.min_crossings > 0: ... col[i] >= variant.min_crossings`
and the same for `min_players` against `players_in_court` ? genuine if/else threshold branches
keyed to SPECIFIC feature names. The served segmenter mirrors this: fusion_hybrid.py:144-151
`_select_for_handoff` loops `if min_crossings > 0 and st.get('net_crossings',0.0) <
min_crossings: continue` / `if person_on and min_players > 0 and ... players_in_court <
min_players`. So the same cue (net_crossings) can enter the decision EITHER as a learned
weighted column (clean) OR via a hard-coded if/else gate. This is by-design
(MULTI_SIGNAL_FUSION_PLAN §3.1 'Decision-level gate, safe baseline, ships first, no labels' vs
§3.2 'Feature-level, primary, the learned path') and is default-OFF (thresholds 0), so it is NOT
a smuggled special-case ? but it means the literal claim 'never an if/else in the decision path'
is false. The gate is a parallel, label-free, hand-set knob, not part of the learned scorer.",
      "The measured + RECOMMENDED win is the if/else gate, not the learned column. current-
state.md:53/70 record Gate-A (net_crossings>=2) as the measured win (dF1 +0.074, 6/6, p=0.031)
with the recommendation `indexing.fusion.min_crossings=2`. So the path the project actually
proposes to promote is the hard threshold (§3.1), while the learned-scorer column for the same
signal (P3 person) measured non-significant. The no-special-casing learned seam exists and is
correct, but it is not currently the winning/served mechanism ? worth not overstating it as
'the' decision path.",
      "Gate-on-absent-feature silently maps to 0 and FAILS the gate (predict drops the window):
experiment.py:186-188 logs a warning then returns zeros, and test_experiment.py:111-114 asserts
an absent `net_crossings` -> 0 -> window dropped by a >0 gate. For the LEARNED path a zero
column is benign (just a feature value), but for the GATE path a missing persisted feature
silently turns a keep into a drop. Minor data-hygiene footgun, not a special-casing issue.",
      "spec_hash (experiment.py:148-153) includes feature_names/min_crossings/min_players, so a
recipe change is traceable ? good ? but the served production gate is driven by
IndexingConfig.fusion (config.json), a SEPARATE surface from the Variant. The offline Variant
and the served `_select_for_handoff` re-implement the same gate logic in two places
(experiment.py:200-210 vs fusion_hybrid.py:144-151); they currently agree but could drift since
there is no shared helper. Not a special-casing defect, but a duplication risk worth noting."
    ]
  },
  {
    "component": "fusion-pipeline files + experiment leaderboard on origin/master",
    "present": false,
    "default_off": true,
    "no_special_casing": true,
    "retained": true,
    "evidence": [
      "Branch/remote state: `git branch --show-current` => docs/i3-fusion-measured; `git log
origin/master -1` => 8836dfe chore: scripts/ ... (#138); `git rev-list --left-right --count
origin/master...HEAD` => 1\t1 (diverged by one commit each side, nothing deleted).",
      "All 9 source files tracked (`git ls-files`):
backend/eval/{experiment,fusion_audio,fusion_compare,fusion_golden}.py,
backend/pipeline/audio/hit_detector.py,
backend/pipeline/detectors/{audio_features,fusion_features,person_detector}.py,
```

```
backend/pipeline/segmenters/fusion_hybrid.py.",
      "All 9 present in origin/master tree (`git ls-tree -r --name-only origin/master | grep
...`) ? each path returned plus its test (test_audio_features.py, test_fusion_compare.py,
test_fusion_features.py, test_fusion_hybrid.py, test_hit_detector.py,
test_person_detector.py).",
      "Per-file last commit on origin/master (`git log origin/master -1 -- <file>`):
person_detector.py=351a27e(#125); fusion_features.py=cbf2236(#117);
audio_features.py=dc19bf2(#122); hit_detector.py=df9cf80(#93); fusion_hybrid.py=cbf2236(#117);
fusion_golden.py=813d1f8(#126); fusion_compare.py=d7b1f51(#127); fusion_audio.py=d7b1f51(#127);
experiment.py=2e5892e(#130).",
      "eval_baselines/experiment_leaderboard.json is NOT tracked: `git ls-files` returns
nothing; `git cat-file -e origin/master:eval_baselines/experiment_leaderboard.json` => fatal:
'exists on disk, but not in origin/master'; same for HEAD; `git status --short` => `??
eval_baselines/experiment_leaderboard.json` (untracked, not gitignored ? `git check-ignore`
returned nothing).",
      "The leaderboard file DOES exist on disk (2506 bytes, mtime Jun 14 11:54) and DOES contain
both rows: P3 = variant_a `shuttle_only` (f1 0.3069) vs variant_b `shuttle_person_fusion` (f1
0.2863), delta f1 -0.0205; P4 = variant_a `shuttle_person` (f1 0.2863) vs variant_b
`shuttle_person_audio` (f1 0.3656), delta f1 +0.0792; both include honest_delta_f1 (bootstrap CI
+ sign test).",
      "Other eval_baselines/ files ARE tracked (gemini_wrapper_2026-06-10.json,
heuristic_lovo_n6.json) ? only the leaderboard JSON is the untracked exception; nothing under
eval_baselines was deleted."
    ],
    "concerns": [
      "present=false ONLY because eval_baselines/experiment_leaderboard.json is not committed to
git (untracked working-tree file). All 9 source/eval files ARE tracked and present on
origin/master, so if the leaderboard's git-tracking is not required, the source-file portion of
the check fully passes.",
      "The leaderboard JSON's content is correct and complete (both P3 and P4 fusion rows with
honest delta-F1), it is simply not yet under version control ? recommend `git add
eval_baselines/experiment_leaderboard.json` and commit if it is meant to be a recorded baseline,
since the sibling eval_baselines JSONs are tracked.",
      "Current checkout is on branch docs/i3-fusion-measured (1 ahead / 1 behind origin/master)
? diverged, not fast-forwardable. Nothing was deleted, but local is not pointed straight at the
remote default branch HEAD.",
      "Measured results are honest: P3 fusion is a slight regression (delta f1 -0.0205, sign
test 2 wins/4 losses, CI includes zero); P4 audio is a positive lift (delta f1 +0.0792) but CI
still includes zero ? neither is statistically significant at n=6, consistent with the default-
OFF posture."
    ]
  },
  {
    "component": "Multi-signal fusion (shuttle + person + audio) cue ? feature producers, the
fusion_hybrid segmenter, and the offline experiment-harness scoring path",
    "present": true,
    "default_off": true,
    "no_special_casing": true,
    "retained": true,
    "evidence": [
      "PRESENT ? producers exist: backend/pipeline/detectors/fusion_features.py:194-215 defines
PERSON_FEATURE_NAMES (5 cols) + compute_person_features() returning exactly those keys;
backend/pipeline/detectors/audio_features.py:15 AUDIO_FEATURE_NAMES (3 cols) +
compute_audio_features(); distill_local.py:40 SHUTTLE FEATURE_NAMES (5 cols).",
      "PRESENT ? served segmenter exists: backend/pipeline/segmenters/fusion_hybrid.py:39 class
FusionSegmenter(TrajectoryHybridSegmenter), registered at :154
segmenter_registry.register(FusionSegmenter). Producers wired in: _enrich_candidate_stats
persists net_crossings always (:85) and person features only when cue_flags.person (:88-89); GPU
person path is lazy (:110-111 'if self._person is None').",
      "PRESENT ? offline measurement path: backend/eval/fusion_golden.py (GPU person extract ->
*_golden_features.json), fusion_compare.py (P3 shuttle vs shuttle+person LOVO), fusion_audio.py
(P4 incremental audio LOVO). All defer to backend/eval/experiment.py::compare (no new scoring
math).",
      "DEFAULT_OFF ? config schema: backend/config/models.py:96 'cue_flags: dict[str,bool] =
Field(default_factory=dict)' (empty => person OFF); :99 min_crossings default 0; :102
```

```
min_players default 0; IndexingConfig.default_segmenter default 'yolo_hybrid' (:112) ? NOT
'fusion'.",
      "DEFAULT_OFF ? on-disk config: config.json has default_segmenter='gemini' and NO 'fusion'
block at all (verified via json load: \"'fusion' in indexing: False\"), so every fusion knob
takes its default-OFF value. fusion_hybrid is opt-in only ? fusion_hybrid.py:22-26 docstring
'Registered as fusion_hybrid, NOT the default segmenter ... Promotion to default happens only on
measured LOVO lift ... never silently.'",
      "DEFAULT_OFF ? even when fusion_hybrid IS selected, default thresholds are no-ops:
_select_for_handoff (fusion_hybrid.py:133-151) only drops windows when min_crossings>0 (:146) or
person_on AND min_players>0 (:148); persisted candidates remain the full superset regardless
(:138 comment).",
      "NO_SPECIAL_CASING ? signals enter as feature COLUMNS the learned LogisticRegression
weights: experiment.py:227-238 predict() builds _feature_matrix(vc, variant.feature_names) and
filters by model.predict_proba >= threshold. The cue is added purely by extending
Variant.feature_names: fusion_compare.py:59-61 fusion variant = SHUTTLE + PERSON names;
fusion_audio.py:82-85 = SHUTTLE + PERSON + AUDIO names. No if/else per-cue branch in the
decision.",
      "NO_SPECIAL_CASING (one caveat) ? the ONLY hard if/else gates are net_crossings (Gate A)
and players_in_court (Gate B) in experiment.py:_gate_mask (:200-210) and
fusion_hybrid.py:_select_for_handoff (:146-148). These are explicit, opt-in, default-0 (OFF)
decision GATES (rally_gate's net-crossing heuristic), distinct from the learned fusion columns;
the person/audio FUSION cue itself is never a hard branch.",
      "RETAINED ? measured n=6 ?F1 was non-significant:
eval_baselines/experiment_leaderboard.json shows person ?F1=-0.021 CI[-0.165,+0.162]
ci_excludes_zero=false (sign 2W/4L p=0.688); audio incr ?F1=+0.079 CI[-0.054,+0.203]
ci_excludes_zero=false. docs/QUALITY_ITERATIONS.md:54 'neither cue clears the promotion bar ...
at n=6 -> both stay default-OFF'; :59-60 'Negative/uncertain results are kept on purpose ? this
is the honest ablation backbone for the paper aim.'",
      "RETAINED ? nothing deleted: producer + feature math still present
(segmenter_registry.register(FusionSegmenter) at fusion_hybrid.py:154 still there). The only
fusion-code change after the n=6 measurement commit (#127) was experiment.py +122/-8 (#130, the
additive C2 threshold-tuning/calibration fix) ? git diff --stat d7b1f51..HEAD shows no
fusion_features.py/fusion_hybrid.py/fusion_compare.py/fusion_audio.py deletions. Re-measurable
by re-running fusion_compare/fusion_audio on more golden videos.",
      "FORWARD-LOOKING SEAM CONFIRMED (do-not-build) ? adding a 'shuttle-on-floor for a
contiguous frame sequence' signal needs exactly the audio-cue pattern, zero special-casing: (1)
NEW backend/pipeline/detectors/shuttle_floor_features.py with SHUTTLE_FLOOR_FEATURE_NAMES +
compute_shuttle_floor_features(points/per_frame, ...) -> dict of those keys (mirrors
fusion_features.py:194/200 and audio_features.py:15/18, both pure/no-IO/[0,1] floats). (2) In
fusion_golden.extract_video (fusion_golden.py:108-115) add a 'shuttle_floor' sub-dict to each
candidate exactly like the 'person' dict; OR follow fusion_audio.augment (fusion_audio.py:43-44)
to add it post-hoc with no GPU. (3) A new Variant whose feature_names = SHUTTLE + PERSON + AUDIO
+ SHUTTLE_FLOOR_FEATURE_NAMES (extend the list at fusion_compare.py:60 / fusion_audio.py:84).
The learned scorer weights it automatically via experiment.py:_feature_matrix (:195-197) ? and
experiment.py:_feature_column (:180-192) already DEFAULTS a missing column to 0.0 with a
warning, so old feature JSONs / DB rows that lack it still vectorise (graceful back-compat). (4)
For SERVED use it would persist into candidate_segments.stats via a new branch in
fusion_hybrid._enrich_candidate_stats gated by a new cue_flags entry (e.g.
cue_flags.shuttle_floor), staying default-OFF like cue_flags.person at fusion_hybrid.py:88. No
decision-path edit required to ADD the column."
    ],
    "concerns": [
      "The two served decision GATES (Gate A net_crossings, Gate B players_in_court) ARE hard-
coded if/else thresholds in BOTH the offline path (experiment.py:_gate_mask :200-210) and
production (fusion_hybrid.py:_select_for_handoff :146-148), not learned-scored. They are
default-OFF (threshold 0) and explicitly the rally_gate heuristic, so they don't violate the no-
special-casing claim for the FUSION cue ? but a future shuttle-floor signal could be mis-
implemented as a third such hard gate (e.g. a min_floor_frames knob) instead of a weighted
feature column. The clean architecture is the feature-column path; flag this so the future
signal is added to feature_names, not as another _gate_mask branch.",
      "Gate A (net_crossings) was actually PROMOTED to a recommended config (min_crossings=2)
per QUALITY_ITERATIONS.md:62-67, but config.json still has NO fusion block, so it is not active
in the shipped served path ? only documented as 'recommended'. Not a default-OFF violation, but
the doc 'promoted' wording slightly overstates shipped state vs. config.json reality.",
      "The person feature producer's REAL-VIDEO path (fusion_hybrid.py:_person_features and
```

PersonDetector.detections_over_windows) is only mock-tested (fusion_hybrid.py:108 '? real-video verification is owner-side; this wiring is mock-tested') and cannot run in this GPU-free container. present/no-special-casing are verified by code reading; the actual GPU feature values feeding the scorer are unverified here.",
      "Audio (P4) ?F1 is measured INCREMENTALLY on top of person, never on shuttle-only directly (fusion_audio.py:79-86), so the audio leaderboard number carries the person 'handicap' (QUALITY_ITERATIONS.md:55-58 notes this and lists it as next work). Doesn't affect retention/default-OFF, but the +0.079 figure is not audio's standalone lift.",
      "Minor: there is also an older 'audio onset detector' that was evaluated and NOT adopted for fusion (per memory note audio-e1-findings); the current fusion_audio.py uses backend/pipeline/audio/hit_detector.detect_hits_in_video ? confirm these aren't conflated when raising N. Not a defect, just lineage to keep straight."
    ]
  }
]

Re-read the actual code and TRY TO REFUTE the overall claim that ALL weak signals (person, audio, court-bounds) are: present, default-OFF, retained (not deleted just because n=6 ?F1 was non-significant), and enter scoring as weighted feature COLUMNS (no special-casing). Specifically hunt for: any signal DELETED/absent; any cue ON by default (cue_flags default True, or default_segmenter set to a fusion family); any hard if/else special-case in the decision path instead of a weighted feature; any code that disables a cue BECAUSE its measured lift was weak; any place a weak signal would be LOST as the corpus grows. Report refuted=true if you find ANY real violation.

2. assistant (2026-06-14T06:35:07.187Z)

I'll re-read the actual code to try to refute the audit's claims. Let me start by pulling the repo fresh and examining the key files.

3. user (2026-06-14T06:35:10.774Z)

```
docs/i3-fusion-measured
9eb06f4 docs(quality): log fusion P3+P4 measured ?F1 on the n=6 golden corpus
1       1
```

4. user (2026-06-14T06:35:10.942Z)

```
backend/eval/classifier.py
backend/eval/distill_local.py
backend/eval/experiment.py
backend/eval/fusion_audio.py
backend/eval/fusion_compare.py
backend/eval/fusion_golden.py
backend/pipeline/audio/hit_detector.py
backend/pipeline/detectors/audio_features.py
backend/pipeline/detectors/fusion_features.py
backend/pipeline/detectors/person_detector.py
backend/pipeline/detectors/rally_gate.py
backend/pipeline/segmenters/fusion_hybrid.py
tests/test_audio_features.py
tests/test_distill_local.py
tests/test_eval_classifier.py
tests/test_experiment.py
tests/test_fusion_compare.py
tests/test_fusion_features.py
tests/test_fusion_hybrid.py
tests/test_hit_detector.py
tests/test_person_detector.py
tests/test_rally_gate.py
```

5. assistant (2026-06-14T06:35:16.156Z)

All 9 source files plus tests are tracked. Let me read the core decision-path files to verify

the no-special-casing and default-OFF claims.

6. user (2026-06-14T06:35:17.579Z)

```
75          min_free_gb: float = 0.0
76
77
78      class FusionConfig(BaseModel):
79          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
80
81          Every default reproduces control behaviour: the fusion segmenter persists the
82          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
83          true ? the person-cue features, but it does NOT gate the served handoff unless a
84          threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
85          is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
86          supplied ? off-court persons (spectators / adjacent courts) are excluded.
87          """
88          # Which trajectory substrate seeds the windows (shuttle stays primary).
89          substrate: Literal["wasb", "tracknet"] = "wasb"
90          # Windowing velocity threshold for the fusion family (the engine reads
91          # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
92          # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
93          velocity_threshold: float = 0.02
94          # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
95          # enabled ? so the default path never imports torch / touches the GPU.
96          cue_flags: dict[str, bool] = Field(default_factory=dict)
97          # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
98          # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
99          min_crossings: int = Field(default=0, ge=0)
100         # Served Gate B: when the person cue is on, drop windows with median in-court
players
101         # < this. 0 = OFF.
102         min_players: int = Field(default=0, ge=0)
103         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
104         court_polygon: Optional[list[list[float]]] = None
105         # Net line for the person two-sided-motion split (person features only ? the shuttle
106         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
107         net_axis: Literal["x", "y"] = "y"
108         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
109
110
111     class IndexingConfig(BaseModel):
112         default_segmenter: str = "yolo_hybrid"
113         use_gpu: bool = True
114         motion_threshold: float = 0.08
115         min_segment_duration: float = 3.0
116         dead_time_merge_gap: float = 2.0
117         chunk_duration: float = 90.0
118         chunk_overlap: float = 10.0
119         detector_model: str = "fasterrcnn_resnet50_fpn"
120         detector_score_threshold: float = 0.5
121         yolo_velocity_threshold: float = 0.15
122         yolo_frame_skip: int = 3
123         yolo_tracker: str = "bytetrack.yaml"
124         yolo_prefetch_workers: int = 2
125         min_action_window_duration: float = 2.0
126         log_yolo_candidates: bool = True
127         store_yolo_candidates: bool = True
128         ai_handoff_padding: float = 2.0
```

```
        ai_max_workers: int = 5
        # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
        # chunks of a high-bitrate (4K) source blow past the provider upload cap
        # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
        # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
        # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
        # Matches gemini_refine's --clip-scale-width default.
        ai_chunk_scale_width: int = 1280
        # Optional debug knob: trim every video to the first N seconds before processing.
        debug_duration: Optional[float] = None

        tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
        wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
        fusion: FusionConfig = Field(default_factory=FusionConfig)

        @field_validator("default_segmenter")
        @classmethod
        def segmenter_must_be_registered(cls, v: str) -> str:
            # Registry check happens at runtime in main/segmenter selection.
            # Here we just allow any string for forward compatibility.
            return v
```

7. user (2026-06-14T06:35:17.932Z)

```
      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).

      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
      shuttle-seeded window via two overridable seams the engine exposes:

      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
        GPU-free, computed from the trajectory we already have); and, only when the person cue
        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
        **Persisted candidates remain the full superset** regardless, so the offline
experiment
        harness can re-score any variant.

      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
      experiment harness (EXPERIMENT_HARNESS.md), never silently.
      """
      from typing import Any, Dict, List, Optional, Tuple

      from backend.pipeline.segmenters.base import segmenter_registry
      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
```

```
    from backend.pipeline.detectors.base import DetectorRunner
    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
    from backend.pipeline.detectors import rally_gate
    from backend.pipeline.detectors import fusion_features as ff


    class FusionSegmenter(TrajectoryHybridSegmenter):
        """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate B)."""

        family = "fusion"
        display_label = "Fusion"

        def __init__(self, db: Any, config: Any):
            super().__init__(db, config)
            # Built lazily only if the person cue is enabled (no detector model loaded otherwise).
            self._person: Optional[Any] = None
            # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-estimate it
            # over the whole trajectory once per candidate window (it depends only on `points`).
            self._axis_cache_key: Optional[int] = None
            self._axis_cache: Optional[tuple] = None

        @property
        def name(self) -> str:
            return "fusion_hybrid"

        @property
        def description(self) -> str:
            return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate rallies, "
                    "net-crossing Gate A + optional person-occupancy Gate B adjudicate them, and the "
                    "AI provider sets semantic boundaries.")

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
            substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
            if substrate == "tracknet":
                cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
                return TrackNetRunner(cfg), {
                    "weights_path": cfg.weights_path,
                    "queue_length": cfg.queue_length,
                    "fusion_substrate": "tracknet",
                }
            wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
            return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate": "wasb"}

        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            stats = super()._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
            # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always persisted
            # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-estimated
            # over the whole trajectory by rally_gate (camera-agnostic); reused across windows.
            gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
                                            min_crossings=0,
```

```
estimate=self._axis_estimate(points))
85              stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87              fcfg = idx_cfg.get("fusion", {})
88              if fcfg.get("cue_flags", {}).get("person", False):
89                  stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90              return stats
91
92          def _axis_estimate(self, points: List[Any]) -> tuple:
93              """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
94              computed once per video, not once per candidate window (it depends only on
points)."""
95              key = id(points)
96              if self._axis_cache_key != key or self._axis_cache is None:
97                  self._axis_cache_key = key
98                  self._axis_cache = rally_gate.estimate_play_axis(points)
99              est = self._axis_cache
100             assert est is not None
101             return est
102
103         def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
104                              frame_width: float, video_path: str,
105                              fcfg: Dict[str, Any]) -> Dict[str, float]:
106             """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
107             frame range and compute the scale/translation-invariant person features. Lazily
builds
108             ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
109             # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
110             from backend.pipeline.detectors.person_detector import PersonDetector
111             if self._person is None:
112                 self._person = PersonDetector(self.config)
113
114             frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
115             # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
116             # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
117             start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
118             det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
119             fw = det["frame_width"] or frame_width
120             fh = det["frame_height"]
121             # If we can't trust the frame dims, every normalised feature would silently
collapse
122             # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
123             if fw <= 0 or fh <= 0:
124                 return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
125
126             poly = fcfg.get("court_polygon")
127             polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
128             net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
129             shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
130             return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
131                                               court_polygon=polygon, net=net)
132
133         def _select_for_handoff(self, windows: List[Dict[str, Any]],
134                                 window_stats: List[Dict[str, Any]],
135                                 idx_cfg: Dict[str, Any]) -> List[int]:
136             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
137             (0) keep every window ? identical to the substrate. Persisted candidates are
```

```
            untouched (full superset)."""
            fcfg = idx_cfg.get("fusion", {})
            min_crossings = int(fcfg.get("min_crossings", 0))
            min_players = int(fcfg.get("min_players", 0))
            person_on = bool(fcfg.get("cue_flags", {}).get("person", False))

            keep: List[int] = []
            for i, st in enumerate(window_stats):
                if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
                    continue
                if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
                    continue
                keep.append(i)
            return keep


    segmenter_registry.register(FusionSegmenter)

```

8. assistant (2026-06-14T06:35:30.212Z)

Confirmed: `cue_flags` defaults empty (person OFF), `min_crossings`/`min_players` default 0 (gates OFF), `court_polygon` default None, `default_segmenter="yolo_hybrid"` (not fusion). The served gates are hard if/else but default-OFF. Now let me verify the learned-scorer decision path and the feature-column claim.

9. user (2026-06-14T06:35:30.615Z)

```
        intervals: List[Interval]
        features: Dict[str, List[float]] = field(default_factory=dict)
        gts: Optional[List[Interval]] = None


    def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
        """Persisted feature column, defaulting missing/short columns to 0.0 (matches
        `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
        col = vc.features.get(name)
        n = len(vc.intervals)
        if col is None:
            logger.warning("variant references feature %r absent from %s ? treating as 0.0",
                           name, vc.name)
            return [0.0] * n
        if len(col) != n:
            raise ExperimentError(
                f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
        return [float(x) for x in col]


    def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
        cols = [_feature_column(vc, name) for name in feature_names]
        return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]


    def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
        """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
        n = len(vc.intervals)
        mask = [True] * n
        if variant.min_crossings > 0:
            col = _feature_column(vc, "net_crossings")
            mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
        if variant.min_players > 0:
            col = _feature_column(vc, "players_in_court")
```

```
            mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
        return mask


    def predict(variant: Variant, vc: VideoCandidates,
                model: Optional[LogisticRegression] = None,
                threshold: Optional[float] = None,
                calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
        """Variant prediction: gate by persisted features, optionally filter by a learned
        model, then merge. Pure and deterministic.

        ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
        fold; if omitted and the variant pins ``classifier_model``, that frozen model is
        loaded. A learned variant with neither a supplied nor a pinned model raises ? there
        is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
        fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
        """
        mask = _gate_mask(variant, vc)
        if variant.feature_names:
            if model is None:
                if variant.classifier_model is None:
                    raise ExperimentError(
                        f"variant {variant.name!r} is learned but has no model to predict "
                        f"with (pass a fitted model or set classifier_model)")
                model = LogisticRegression.load(variant.classifier_model)
            proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
            if calibrator is not None:
                proba = [calibrator(p) for p in proba]
            thr = variant.threshold if threshold is None else threshold
            mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
        kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
        return merge_overlaps(kept, gap_tol=variant.merge_gap)


    def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
                            train_videos: Sequence[VideoCandidates], iou: float
                            ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
        """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
        (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
        (use the variant default). Honest: never looks at the held-out video.
        """
        calibrator: Optional[Callable[[float], float]] = None
        if variant.calibrate:
            raw: List[float] = []
            y: List[int] = []
            for vc in train_videos:
                raw.extend(float(p) for p in
                           model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
                y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
            if variant.calibrate == "platt":
                calibrator = fit_platt(raw, y)
            elif variant.calibrate == "isotonic":
                calibrator = fit_isotonic(raw, y)
            else:
                raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
        threshold: Optional[float] = None
        if variant.tune_threshold:
            best_t, best_f1 = variant.threshold, -1.0
            for k in range(1, 20):                      # grid 0.05..0.95 on the train
fold's rally F1
```

```
268                 t = round(0.05 * k, 2)
269                 f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
270                              vc.gts or [], iou).f1 for vc in train_videos]
271                 mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
272                 if mean_f1 > best_f1:
273                     best_f1, best_t = mean_f1, t
274             threshold = best_t
275         return calibrator, threshold
276
277
278     # ---------------------------------------------------------------------- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`."""
284         X: List[List[float]] = []
285         y: List[int] = []
286         for vc in videos:
287             if vc.gts is None:
288                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289             X.extend(_feature_matrix(vc, variant.feature_names or []))
290             y.extend(label_candidates(vc.intervals, vc.gts, iou))
291         if not X:
292             raise ExperimentError("cannot train a learned variant on zero candidates")
293         return LogisticRegression().fit(X, y, variant.feature_names)
294
```

10. user (2026-06-14T06:35:30.968Z)

```
185             aligned += 1
186             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
187                    for _tid, box in dets):
188                 colocated += 1
189         return colocated / aligned if aligned else 0.0
190
191
192     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194     PERSON_FEATURE_NAMES = [
195         "players_in_court", "two_sided_motion", "mean_player_motion",
196         "in_court_ratio", "shuttle_player_colocation",
197     ]
198
199
200     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
201                                 frame_width: float, frame_height: float,
202                                 court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
203                                 net: Optional[Net] = None,
204                                 min_players: int = 2) -> Dict[str, float]:
205         """Aggregate the per-window person features from per-frame detections + the in-
window
206         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
207         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
208         return {
209             "players_in_court": players_in_court(counts),
210             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
211             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212             "in_court_ratio": in_court_ratio(counts, min_players),
```

```
213             "shuttle_player_colocation": shuttle_player_colocation(
214                 per_frame, shuttle_points, frame_width, frame_height),
215         }
216
```

11. assistant (2026-06-14T06:35:37.359Z)

The decision path confirms: learned variants enter via `feature_names` columns through `_feature_matrix` ? `predict_proba`. The only hard if/else gates are `_gate_mask` (net_crossings, players_in_court), both default-OFF (`min_crossings`/`min_players` default 0). Let me verify the audio cue path and the Variant defaults, plus confirm no cue is disabled because of weak measured lift.

12. user (2026-06-14T06:35:39.228Z)

```
80          A variant turns one video's persisted candidate windows + features into rally
81          intervals. Every knob defaults to the **control** behaviour (no gate, no learned
82          filter), so a variant that merely *carries* a new, disabled cue is byte-identical
83          to control ? the core no-regression guarantee.
84
85          Fields:
86            - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87              for the leaderboard and for selecting the served path (the existing
88              `segmenter_registry` + `IndexingConfig` do the actual selection in production).
89              New cues are recorded here default-OFF.
90            - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
91              ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92              `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93              windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94            - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
95              whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96              cue persists that feature).
97            - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
98              `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
99              required for `compare_unlabeled` on a learned variant.
100           - ``feature_names`` ? the persisted features the learned filter consumes. When set
101             WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
102             (honest) ? the recipe, not a pinned artifact.
103           - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104             overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
105         """
106
107         name: str
108         segmenter: str = "wasb_hybrid"
109         config_overrides: Dict[str, Any] = field(default_factory=dict)
110         cue_flags: Dict[str, bool] = field(default_factory=dict)
111         min_crossings: int = 0
112         min_players: int = 0
113         classifier_model: Optional[str] = None
114         feature_names: Optional[List[str]] = None
115         threshold: float = 0.5
116         merge_gap: float = 1.0
117         # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118         # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119         # failure where a fixed 0.5 zeroed out a small-candidate video.
120         tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121         calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first
122
123         def is_learned(self) -> bool:
124             """True if this variant applies a learned classifier (pinned model or
recipe)."""
```

```
            return self.classifier_model is not None or bool(self.feature_names)

        def to_dict(self) -> dict:
            return {
                "name": self.name,
                "segmenter": self.segmenter,
                "config_overrides": self.config_overrides,
                "cue_flags": self.cue_flags,
                "min_crossings": self.min_crossings,
                "min_players": self.min_players,
                "classifier_model": self.classifier_model,
                "feature_names": self.feature_names,
                "threshold": self.threshold,
                "merge_gap": self.merge_gap,
                "tune_threshold": self.tune_threshold,
                "calibrate": self.calibrate,
            }

        @classmethod
        def from_dict(cls, d: dict) -> "Variant":
            known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
            return cls(**{k: v for k, v in d.items() if k in known})

        def spec_hash(self) -> str:
            """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
            and makes a result reproducible. The pinned **control** variant always hashes
the
            same, so a regression is always traceable to a recipe change, never to drift."""
            blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
            return sha1(blob.encode()).hexdigest()[:12]
```

13. user (2026-06-14T06:35:39.498Z)

```
"""Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-
only).

Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment`
runs
`detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio
features per
window; `compare_increment` then runs the SHIPPED harness between a shuttle+person
variant
and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON
TOP
of person, not against raw shuttle. Audio enters only as columns the learned scorer
weights
(no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.

Run (no GPU):
    python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json
--features-dir output --augment --compare
"""
from __future__ import annotations

import argparse
import glob
import json
import os
from typing import Any, Dict, List

from backend.eval import experiment
from backend.eval.fusion_compare import delta_stats
from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES,
_derive_video_path
```

```
    from backend.pipeline.detectors import audio_features as af
    from backend.pipeline.audio.hit_detector import detect_hits_in_video


    def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
        """Add the 3 audio features to each existing ``<name>_golden_features.json``
window."""
        with open(manifest_path, encoding="utf-8") as f:
            specs = json.load(f)
        written: List[str] = []
        for spec in specs:
            jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
            if not os.path.isfile(jp):
                print(f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden
first; skipping")
                continue
            video = _derive_video_path(spec)
            print(f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
flush=True)
            hits = detect_hits_in_video(video, k=k)
            with open(jp, encoding="utf-8") as f:
                data = json.load(f)
            for c in data["candidates"]:
                c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
            with open(jp, "w", encoding="utf-8") as f:
                json.dump(data, f, indent=2)
            print(f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows", flush=True)
            written.append(jp)
        return written


    def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
        """Load the feature JSONs into VideoCandidates including the audio columns (requires
        `augment` to have run ? raises if a JSON lacks audio)."""
        videos: List[experiment.VideoCandidates] = []
        for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
            with open(path, encoding="utf-8") as f:
                d = json.load(f)
            cands = d["candidates"]
            if cands and "audio" not in cands[0]:
                raise SystemExit(f"{path} has no audio features ? run `--augment` first")
            intervals = [(float(c["start"]), float(c["end"])) for c in cands]
            features: Dict[str, List[float]] = {}
            for fn in SHUTTLE_FEATURE_NAMES:
                features[fn] = [float(c["shuttle"][fn]) for c in cands]
            for fn in PERSON_FEATURE_NAMES:
                features[fn] = [float(c["person"][fn]) for c in cands]
            for fn in af.AUDIO_FEATURE_NAMES:
                features[fn] = [float(c["audio"][fn]) for c in cands]
            gts = [(float(a), float(b)) for a, b in d["gts"]]
            videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
                                                     features=features, gts=gts))
        return videos


    def compare_increment(videos: List[experiment.VideoCandidates], iou: float = 0.5,
                          threshold: float = 0.5) -> Dict[str, Any]:
        """Incremental audio lift: shuttle+person  vs  shuttle+person+audio (honest
LOVO)."""
        person = experiment.Variant(
            name="shuttle_person",
            feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES),
threshold=threshold)
        person_audio = experiment.Variant(
```

```
83              name="shuttle_person_audio",
84              feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES) +
list(af.AUDIO_FEATURE_NAMES),
85              threshold=threshold)
86          return experiment.compare(videos, person, person_audio, iou=iou, honest=True)
87
88
89      def format_result(res: Dict[str, Any], iou: float) -> str:
90          a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
91          ds = delta_stats(res)
92          st = ds["sign_test"]
93          lines = [
94              f"=== Fusion P4 incremental AUDIO LOVO (n={res['num_videos']}, IoU>={iou},
honest) ===",
95              f"  shuttle+person       : F1={a['f1']:.3f}  P={a['precision']:.3f}
R={a['recall']:.3f}",
96              f"  shuttle+person+audio : F1={b['f1']:.3f}  P={b['precision']:.3f}
R={b['recall']:.3f}",
97              f"  incremental ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)",
98              f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
99              f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
100             f"| CI clears 0: {ds['ci_excludes_zero']}",
101         ]
102         for pv in res["per_video_delta"]:
103             lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
104         return "\n".join(lines)
105
106
107     def main() -> None:
108         ap = argparse.ArgumentParser(description="Fusion P4 ? incremental audio lift on the
golden set.")
109         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
110         ap.add_argument("--features-dir", default="output")
111         ap.add_argument("--k", type=float, default=2.5, help="hit-detector sensitivity")
112         ap.add_argument("--augment", action="store_true", help="add audio features to the
existing JSONs")
113         ap.add_argument("--compare", action="store_true", help="report the incremental audio
LOVO ?F1")
114         ap.add_argument("--iou", type=float, default=0.5)
115         ap.add_argument("--record", action="store_true")
116         args = ap.parse_args()
117
118         if args.augment:
119             augment(args.manifest, args.features_dir, args.k)
120         if args.compare:
121             videos = load_videos_with_audio(args.features_dir)
122             if len(videos) < 2:
123                 raise SystemExit(f"need >=2 augmented feature files in {args.features_dir}")
124             res = compare_increment(videos, iou=args.iou)
125             print(format_result(res, args.iou))
126             if args.record:
127                 row = experiment.record_experiment(res)
128                 print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']})")
129
130
131     if __name__ == "__main__":
132         main()
133
```

14. assistant (2026-06-14T06:35:45.534Z)

Audio enters purely as feature columns (`SHUTTLE + PERSON + AUDIO` names) through the same learned scorer. No hard if/else on audio anywhere. Now let me check the critical "lost as corpus

grows" claim ? the `_feature_column` back-compat for missing columns, and the `load_videos` paths in `fusion_compare.py` and `fusion_golden.py`. Let me also confirm git history shows no deletion/disable-due-to-weak-lift commits.

15. user (2026-06-14T06:35:46.976Z)

```
1       """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
set.
2
3       The fusion-P3 measurement. Loads the per-video feature JSONs written by
`fusion_golden.py`
4       and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
out) ?
5       no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
harness's
6       designed seam): the person features are extra columns the learned LogisticRegression
weights,
7       so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9       Run (no GPU):
10          python -m backend.eval.fusion_compare --features-dir output --record
11      """
12      from __future__ import annotations
13
14      import argparse
15      import glob
16      import json
17      import os
18      from typing import Any, Dict, List
19
20      from backend.eval import experiment
21      from backend.eval.eval_stats import paired_delta_ci
22      from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25      def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26          """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27          (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28          by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29          by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30          by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31          vids = [p["video"] for p in res["variant_a"]["per_video"]]
32          return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35      def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36          """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37          feature columns + golden gts). The harness derives labels from gts itself."""
38          videos: List[experiment.VideoCandidates] = []
39          for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40              with open(path, encoding="utf-8") as f:
41                  d = json.load(f)
42              cands = d["candidates"]
43              intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44              features: Dict[str, List[float]] = {}
45              for fn in SHUTTLE_FEATURE_NAMES:
46                  features[fn] = [float(c["shuttle"][fn]) for c in cands]
47              for fn in PERSON_FEATURE_NAMES:
48                  features[fn] = [float(c["person"][fn]) for c in cands]
49              gts = [(float(a), float(b)) for a, b in d["gts"]]
50              videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                                                       features=features, gts=gts))
```

```
        return videos


    def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
                threshold: float = 0.5) -> Dict[str, Any]:
        shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
                                          threshold=threshold)
        fusion = experiment.Variant(name="shuttle_person_fusion",
                                    feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),
                                    threshold=threshold)
        return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)


    def format_result(res: Dict[str, Any], iou: float) -> str:
        a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
        ds = delta_stats(res)
        st = ds["sign_test"]
        lines = [
            f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
===",
            f"  shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
            f"  +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
            f"  ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)
?P={d['precision']:+.3f}  ?R={d['recall']:+.3f}",
            f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
            f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
            f"| CI clears 0: {ds['ci_excludes_zero']}",
            "  per-video ?F1:",
        ]
        for pv in res["per_video_delta"]:
            lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
        return "\n".join(lines)


    def main() -> None:
        ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
shuttle+person (golden set).")
        ap.add_argument("--features-dir", default="output")
        ap.add_argument("--iou", type=float, default=0.5)
        ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
        args = ap.parse_args()

        videos = load_videos(args.features_dir)
        if len(videos) < 2:
            raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
        res = compare(videos, iou=args.iou)
        print(format_result(res, args.iou))
        if args.record:
            row = experiment.record_experiment(res)
            print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']})")


    if __name__ == "__main__":
        main()

```

16. user (2026-06-14T06:35:47.597Z)

```
95          candidates: List[Dict[str, Any]] = []
96          for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
97              det = per_window[i]
98              det_fw = det.get("frame_width") or fw
99              det_fh = det.get("frame_height") or 0.0
100             # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
101             # normalisation fusion_features uses (x/width, y/height).
102             if axis == "x":
103                 net_norm = net_px / det_fw if det_fw > 0 else 0.5
104             else:
105                 net_norm = net_px / det_fh if det_fh > 0 else 0.5
106             sf, ef = win_frames[i]
107             shuttle_in = [p for p in points if sf <= p.frame <= ef]
108             person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
109                                                 court_polygon=None, net=(axis, net_norm))
110             candidates.append({
111                 "start": float(s), "end": float(e),
112                 "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
113                 "person": {k: float(v) for k, v in person.items()},
114                 "label": int(labels[i]),
115             })
116         return {
117             "name": spec["name"], "fps": fps, "frame_width": fw,
118             "candidates": candidates,
119             "gts": [[float(a), float(b)] for a, b in gts],
120         }
121
122
123     def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
124             fresh: bool = False, smallest_first: bool = False) -> List[str]:
125         with open(manifest_path, encoding="utf-8") as f:
126             specs = json.load(f)
127         if smallest_first:
128             # Process small videos first for quick feedback (the slow full match lands
last);
129             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
130             specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
131                                                  if os.path.isfile(s.get("trajectory", ""))
else 0))
132         os.makedirs(out_dir, exist_ok=True)
133         written: List[str] = []
134         for vi, spec in enumerate(specs, 1):
```

17. assistant (2026-06-14T06:35:52.232Z)

I notice a subtle point in `fusion_compare.load_videos` (lines 47-48): it does `float(c["person"][fn])` directly ? meaning it assumes every candidate has a `person` dict. This is the "lost as corpus grows" angle worth checking. But the key claim is about the learned scorer's `_feature_column` graceful default. Let me verify the audio feature math, check git history for any deletion/disable commits, and verify the test asserting disabled cues are byte-identical.

18. user (2026-06-14T06:35:53.187Z)

```
1       """Pure per-window audio-cue features for rally fusion (fusion P4).
2
3       NO I/O ? the audio hit detection lives in `backend/pipeline/audio/hit_detector.py`
4       (`detect_hits_in_video` -> `List[HitEvent]`); this only turns those hit events into a
few
5       per-window features the learned scorer can weight. Mirrors `fusion_features.py`
discipline.
6
7       Honest expectation (see the `audio-e1-findings` memory): the classical-onset audio cue
```

```
had
8       only ~2.2x discrimination and was NOT adopted standalone ? so this is "add the signal as
a
9       column and let the golden-set harness decide", default-OFF, not "adopt audio".
10      """
11      from __future__ import annotations
12
13      from typing import Dict, List
14
15      AUDIO_FEATURE_NAMES = ["audio_hit_count", "audio_hit_rate", "audio_hit_strength_mean"]
16
17
18      def compute_audio_features(hits: List, window_start: float, window_end: float) ->
Dict[str, float]:
19          """Audio features for one window from a video's HitEvents (each has ``.t``,
``.strength``).
20
21          A real rally is a sustained volley of racket?shuttle contacts; dead time / a single
22          service feed has few. Returns hit count, hits/second, and mean strength ? all 0.0 on
an
23          empty window. ``[start, end)`` half-open so a hit on a boundary belongs to one
window only.
24          """
25          dur = max(1e-6, window_end - window_start)
26          in_win = [h for h in (hits or []) if window_start <= h.t < window_end]
27          if not in_win:
28              return {k: 0.0 for k in AUDIO_FEATURE_NAMES}
29          n = len(in_win)
30          return {
31              "audio_hit_count": float(n),
32              "audio_hit_rate": n / dur,
33              "audio_hit_strength_mean": sum(h.strength for h in in_win) / n,
34          }
35
```

19. user (2026-06-14T06:35:56.475Z)

```
=== deletions of cue files ===
=== commits mentioning disable/remove cue ===
ecee786 feat(eval): experiment harness ? A/B + SxS variant layer (no-regression by design)
(#114)
6de70aa fix(stitcher): escape drive-letter colon in ffmpeg filtergraph paths (#108)
10864f3 feat(stitcher): configurable branding watermark on highlight reels (#101)
ada4ca7 feat(stitcher): configurable branding watermark on highlight reels
f01387e test(reporting): pin the CI-visible half of the oversize channel (no obsreport needed)
(#82)
3377486 test(reporting): pin the CI-visible half of the oversize channel (no obsreport needed)
(#81)
a2e65d7 fix(gemini): re-encode AI chunks (4K oversize skip = silent 0-segments) + loud report
flag (#80)
2477136 feat(reports): extend for Input Quality UX (audio readiness + capture guidance) ?
scaffolding item 2
6becb07 feat: persist and log raw YOLO candidate rallies for fine-tuning
5257fd5 feat: pipelined YOLO chunk processing & unified Gemini model config
0b79ab6 Optimize Gemini segmenter with automatic low-bitrate compression for large videos,
disable config debug_duration by default, and update stitcher output filename
=== END ===
```

20. assistant (2026-06-14T06:36:05.435Z)

```
No cue-file deletions in history. The grep matches are unrelated (the "disable" hits are about
debug_duration, watermarks, etc., not cue disabling). The audio feature math is pure numeric, no
special-casing. Now let me verify the no-regression test and the LogisticRegression scorer is
genuinely cue-agnostic (learns one weight per column, no per-cue branch).
```

21. user (2026-06-14T06:36:06.853Z)

```
            self.feature_names: Optional[List[str]] = None

        @staticmethod
        def _sigmoid(z: np.ndarray) -> np.ndarray:
            # Clip for numerical stability (avoid overflow in exp).
            return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))

        def _standardize(self, X: np.ndarray) -> np.ndarray:
            return (X - self.mean) / self.std

        def fit(self, X, y, feature_names: Optional[List[str]] = None,
                class_weight: str = "balanced") -> "LogisticRegression":
            """Fit with optional balanced class weighting (rally windows are imbalanced
            ? many spurious candidates per real rally)."""
            X = np.asarray(X, dtype=float)
            y = np.asarray(y, dtype=float)
            if X.ndim != 2 or X.shape[0] == 0:
                raise ValueError("X must be a non-empty 2D array")
            self.feature_names = feature_names

            self.mean = X.mean(axis=0)
            self.std = X.std(axis=0)
            self.std[self.std == 0] = 1.0  # guard constant features
            Xs = self._standardize(X)
            n, d = Xs.shape

            if class_weight == "balanced":
                n_pos = max(1, int(y.sum()))
                n_neg = max(1, int((1 - y).sum()))
                w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
            else:
                w_pos = w_neg = 1.0
            sw = np.where(y == 1, w_pos, w_neg)

            self.w = np.zeros(d)
            self.b = 0.0
            for _ in range(self.epochs):
                p = self._sigmoid(Xs @ self.w + self.b)
                err = (p - y) * sw
                grad_w = Xs.T @ err / n + self.l2 * self.w
                grad_b = float(err.mean())
                self.w -= self.lr * grad_w
                self.b -= self.lr * grad_b
            return self

        def predict_proba(self, X) -> np.ndarray:
            if self.w is None:
                raise RuntimeError("model is not fitted")
            Xs = self._standardize(np.asarray(X, dtype=float))
            return self._sigmoid(Xs @ self.w + self.b)

        def predict(self, X, threshold: float = 0.5) -> np.ndarray:
            return (self.predict_proba(X) >= threshold).astype(int)

        # --- persistence (small JSON, no pickle/joblib) ---------------------------

        def to_dict(self) -> dict:
            return {
                "model": "logreg",
                "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
                "w": self.w.tolist() if self.w is not None else None,
                "b": self.b,
                "mean": self.mean.tolist() if self.mean is not None else None,
```

```
                "std": self.std.tolist() if self.std is not None else None,
                "feature_names": self.feature_names,
            }

        @classmethod
        def from_dict(cls, d: dict) -> "LogisticRegression":
            m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
```

22. user (2026-06-14T06:36:07.426Z)

```
    def test_is_learned():
        assert not CONTROL.is_learned()
        assert Variant(name="r", feature_names=["x"]).is_learned()
        assert Variant(name="p", classifier_model="model.json").is_learned()


    # ------------------------------------------------------ the no-regression invariant

    def test_disabled_cues_byte_identical_to_control():
        """A variant carrying only DISABLED cues must equal control, window-for-window."""
        vc = _held_shuttle_video()
        disabled = Variant(name="fusion_off", cue_flags={"person": False, "audio": False},
                           min_crossings=0, min_players=0)
        assert predict(disabled, vc) == predict(CONTROL, vc)


    def test_control_is_merged_candidates():
        vc = VideoCandidates(name="v", intervals=[(0.0, 10.0), (10.5, 12.0), (40.0, 50.0)])
        # merge_gap=1.0 stitches the 0.5s gap between the first two; the 28s gap stays
split.
        assert predict(CONTROL, vc) == [(0.0, 12.0), (40.0, 50.0)]


    # ------------------------------------------------------------------ decision gates

    def test_gate_a_drops_low_crossing_window():
        vc = _held_shuttle_video()
        gate_a = Variant(name="gateA", min_crossings=2)
        preds = predict(gate_a, vc)
        assert (12.0, 13.0) not in preds                          # held-shuttle fire
removed
        assert preds == [(0.0, 10.0), (20.0, 30.0)]
        assert preds != predict(CONTROL, vc)                       # the gate actually
changed output


    def test_gate_b_filters_on_players():
        vc = VideoCandidates(
            name="v", intervals=[(0.0, 10.0), (20.0, 30.0)],
            features={"players_in_court": [2.0, 1.0]})
        preds = predict(Variant(name="gateB", min_players=2), vc)
        assert preds == [(0.0, 10.0)]                              # the 1-player window is
dropped


    def test_gate_on_absent_feature_defaults_to_zero(caplog):
        vc = VideoCandidates(name="v", intervals=[(0.0, 10.0)])    # no net_crossings
persisted
        preds = predict(Variant(name="gateA", min_crossings=2), vc)
        assert preds == []                                         # absent feature ? 0 ?
fails gate


    # ------------------------------------------------------------- evaluate_variant

```

```
119     def test_evaluate_static_variant_scores_vs_gts():
120         res = evaluate_variant([_held_shuttle_video()], CONTROL, iou=0.5)
121         pv = res["per_video"][0]
122         # control keeps the false fire: 2 TP, 1 FP ? precision 2/3, recall 1.0
123         assert pv["recall"] == pytest.approx(1.0)
124         assert pv["precision"] == pytest.approx(2 / 3)
125         assert res["aggregate"]["recall"] == pytest.approx(1.0)
126         assert not res["honest_lovo"]
127
128
129     def test_evaluate_learned_lovo_recovers_separable_signal():
130         variant = Variant(name="learned", feature_names=["x"], threshold=0.5)
131         res = evaluate_variant(_separable_videos(), variant, iou=0.5, honest=True)
132         assert res["honest_lovo"] is True
133         assert res["aggregate"]["f1"] == pytest.approx(1.0)        # held-out F1 perfect on
separable data
134
135
136     def test_evaluate_learned_lovo_needs_two_videos():
137         variant = Variant(name="learned", feature_names=["x"])
138         with pytest.raises(ExperimentError):
139             evaluate_variant(_separable_videos()[:1], variant, honest=True)
```

23. assistant (2026-06-14T06:36:18.945Z)

The `LogisticRegression` is fully cue-agnostic: it learns one weight per column (`self.w = np.zeros(d)`), with no per-cue branching. The test `test_disabled_cues_byte_identical_to_control` confirms disabled cues produce identical output to control.

Now let me verify the QUALITY_ITERATIONS log says cues are RETAINED despite weak lift (not deleted), and confirm the on-disk config.json has no fusion block. Let me also double-check that no cue is keyed to a hard if/else in the production decision path beyond the documented default-OFF gates.

24. user (2026-06-14T06:36:23.007Z)

```
=== config.json default_segmenter + fusion block ===
default_segmenter: gemini
fusion in indexing: False
=== any if/else on audio_hit / players / net_crossings / cue in non-eval served code ===
backend/pipeline/detectors/audio_features.py:15:AUDIO_FEATURE_NAMES = ["audio_hit_count",
"audio_hit_rate", "audio_hit_strength_mean"]
backend/pipeline/detectors/audio_features.py:31:        "audio_hit_count": float(n),
backend/pipeline/detectors/audio_features.py:32:        "audio_hit_rate": n / dur,
backend/pipeline/detectors/fusion_features.py:97:def players_in_court(counts: Sequence[int]) ->
float:
backend/pipeline/detectors/fusion_features.py:195:    "players_in_court", "two_sided_motion",
"mean_player_motion",
backend/pipeline/detectors/fusion_features.py:209:        "players_in_court":
players_in_court(counts),
backend/pipeline/detectors/rally_gate.py:69:def count_net_crossings(window_points, axis: str,
net: float, deadband: float) -> int:
backend/pipeline/detectors/rally_gate.py:106:        c = count_net_crossings(wp, axis, net,
deadband)
backend/pipeline/segmenters/fusion_hybrid.py:22:Defaults reproduce the substrate exactly:
net_crossings/person features are persisted but
backend/pipeline/segmenters/fusion_hybrid.py:85:        stats["net_crossings"] =
float(gated[0].crossings) if gated else 0.0
backend/pipeline/segmenters/fusion_hybrid.py:146:            if min_crossings > 0 and
st.get("net_crossings", 0.0) < min_crossings:
backend/pipeline/segmenters/fusion_hybrid.py:148:            if person_on and min_players > 0
and st.get("players_in_court", 0.0) < min_players:
backend/pipeline/segmenters/trajectory_hybrid.py:87:        rows; ``FusionSegmenter`` overrides
this to add ``net_crossings`` (+ person-cue
```

backend/pipeline/segmenters/trajectory_hybrid.py:201:   # segmenter adds net_crossings +
person-cue features. Computed once, reused for both

25. user (2026-06-14T06:36:25.535Z)

backend/eval/fusion_golden.py:33:SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
backend/eval/fusion_golden.py:34:# The 5 person-cue features
(fusion_features.PERSON_FEATURE_NAMES).
backend/eval/fusion_golden.py:35:PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
backend/eval/fusion_golden.py:112:   "shuttle": {k: float(v) for k, v in
zip(SHUTTLE_FEATURE_NAMES, xs)},
=== QUALITY_ITERATIONS fusion rows ===
12:graded **offline against golden labels** (per-video + LOVO-honest); promotion to default
happens **only
43:### Fusion P3 + P4 MEASURED on the n=6 golden corpus ? person = no lift, audio = promising
(not promotable) *(2026-06-14)*
44:The GPU person-feature extraction finished all 6 golden videos (`fusion_golden.py` ?
`output/*_golden_features.json`, the
51:| + person (P3) | 0.286 | **?0.021** (?6.7%) | [?0.165, +0.162] | 2W/4L (p=0.688) | **No** |
52:| + person + audio (P4 incr.) | 0.366 | **+0.079** (+27.7%) | [?0.054, +0.203] | 4W/2L
(p=0.688) | **No** |
54:**Read honestly:** neither cue clears the promotion bar (both CIs span 0, both sign tests
n.s.) at n=6 ? **both stay
55:default-OFF** (promote-only-on-lift). **Person** adds noise (no court polygons ?
`players_in_court` counts spectators);
56:per-video it's a wash (testlarge +0.40, gbaaddy ?0.22). **Audio** is the more promising lead
? +0.079 / +27.7%, wins 4/6
57:(largetest +0.27, mahadevpura +0.27), and shuttle+person+audio (0.366) lands **above**
shuttle-only (0.307) ? but n=6 can't
58:confirm it. **Next:** more matches (raise N until the sign test can resolve ~+0.08), and
measure audio on shuttle-only
59:*directly* (without the person handicap). Recorded in
`eval_baselines/experiment_leaderboard.json`. **Negative/uncertain
60:results are kept on purpose ? this is the honest ablation backbone for the paper aim.**
69:(the production fusion segmenter is P2/unverified on real video) ? so this records the
promotion + rationale,
77:additive/default-OFF `Variant.tune_threshold` (operating point chosen on the **train** fold)
+ `calibrate`
93:Next: Gate-A is a candidate for the no-regression promotion gate; the logistic failure is the
C2 (calibration)
96:### Eval-honesty helpers (course corrections C1?C4) ? additive, default-OFF *(2026-06-14)*
107: + `brier_score`/`expected_calibration_error`, so a CNN softmax, an audio onset, and an
integer net-crossing
122:and a person-occupancy **Gate B** raises **precision** on the held-shuttle / warm-up / one-
sided-feed /
142:player-count-only; supply ? masked). Person features (`players_in_court, two_sided_motion,
144:learned fusion. Reuses `PersonDetector` (P1) + `rally_gate` (already existed) ? no
reinvention.
146:**Status (2026-06-14): SHIPPED, default-OFF.** `FusionSegmenter`
(`backend/pipeline/segmenters/fusion_hybrid.py`,
148:identity hooks on the trajectory engine (`_enrich_candidate_stats`, `_select_for_handoff`) +
`PersonDetector.
154:`rally_gate` estimate, honest torch-import docstrings, and 6 new tests closing the person-
detector / Gate-B /
156:**Two findings deferred with intent** (reviewer-agreed): person features are computed
eagerly even for
160:held-shuttle-drop + person-feature values are GPU/owner-side. *(Measured numbers appended
here when run ? no claims before.)*
162:### Person cue extracted to a reusable producer (fusion P1) ?
[#115](https://github.com/avidullu/badminton-highlight-indexer/pull/115), 2026-06-13
164:`backend/pipeline/detectors/person_detector.py::PersonDetector`. **No behavior change**
(proven by the
166:it makes "add the person cue to fusion" an *add-a-producer*, not a segmenter rewrite. Suite
440?441 green.
198: for the same reason (foreground spatial gate **?0.04**; audio energy **AUC ~0.6**; court-

```
region
202:- **Court-masking ? multi-court fix** (see above) ? it's a **precision** tool for off-court
persons and
```

26. assistant (2026-06-14T06:36:31.752Z)

```
All confirmed. The QUALITY_ITERATIONS log explicitly says both cues stay default-OFF and
"Negative/uncertain results are kept on purpose." config.json has no fusion block and ships
`default_segmenter=gemini`. The only hard if/else on cue features in served code are the two
documented default-OFF gates (`min_crossings`/`min_players` both default 0).

Let me do a final check on the one genuine "lost as corpus grows" risk I spotted ? whether
`fusion_compare.load_videos`/`fusion_audio.load_videos_with_audio` would silently drop a
feature, versus the graceful `_feature_column` default-to-0 in the learned scorer. Let me
confirm the experiment harness path (which is what scores) handles missing columns gracefully.
```

27. user (2026-06-14T06:36:33.049Z)

```
95
96
97      def players_in_court(counts: Sequence[int]) -> float:
98          """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99          return _median(list(counts))
100
101
102     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
104         if not counts:
105             return 0.0
106         return sum(1 for c in counts if c >= min_players) / len(counts)
107
108
109     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110                            frame_width: float, frame_height: float) -> float:
111         """Mean per-track centre displacement between consecutive sampled frames, with x
112         normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113         not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114         if frame_width <= 0 or len(per_frame) < 2:
115             return 0.0
116         frames = sorted(per_frame.keys())
117         steps: List[float] = []
118         for a, b in zip(frames, frames[1:]):
119             prev = {tid: box for tid, box in per_frame[a]}
120             for tid, box in per_frame[b]:
121                 if tid in prev:
122                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125         if not steps:
126             return 0.0
127         return sum(steps) / len(steps)
128
129
130     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
131                          frame_height: float, net: Optional[Net],
132                          court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
0.0.
137         """
138         if net is None or not per_frame:
139             return 0.0
```

```
140         axis, pos = net
141         both = 0
142         for _frame_idx, dets in per_frame.items():
143             near, far = False, False
144             for _tid, box in dets:
145                 if not person_in_court(box, frame_width, frame_height, court_polygon):
146                     continue
147                 fx, fy = _foot_norm(box, frame_width, frame_height)
148                 coord = fx if axis == "x" else fy
149                 if coord < pos:
150                     near = True
151                 else:
152                     far = True
153             if near and far:
154                 both += 1
155         return both / len(per_frame)
156
157
158     def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
159                         pad_frac: float) -> bool:
160         """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
161         width/height (the 'adjacent / in-hand' tolerance)?"""
162         x1, y1, x2, y2 = box
163         px = pad_frac * frame_width
164         py = pad_frac * frame_height
165         return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
166
167
168     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
169                                   frame_width: float, frame_height: float,
```

28. assistant (2026-06-14T06:36:52.084Z)

All feature math is pure numeric, returns floats, no hard keep/drop decisions. I have thoroughly verified every claim. Let me compile my findings.

Summary of verification against the four sub-claims for all three weak signals (person, audio, court-bounds):

Present ? All producers tracked and intact: `fusion_features.py` (person + court math, `PERSON_FEATURE_NAMES` at :194, `point_in_polygon`/`person_in_court`/`in_court_counts`), `audio_features.py` (`AUDIO_FEATURE_NAMES` at :15, `compute_audio_features`), `person_detector.py`, `hit_detector.py`, plus eval drivers `fusion_compare.py`/`fusion_audio.py`/`fusion_golden.py`. `git log --diff-filter=D` over all cue files = empty. No deletion ever.

Default-OFF ? Confirmed at every layer: `cue_flags` defaults `{}` (person OFF, models.py:96), `min_crossings`/`min_players` default 0 (models.py:99/102), `court_polygon` default None (models.py:104), `default_segmenter="yolo_hybrid"` (models.py:112), shipped config.json ships `gemini` with NO fusion block at all (verified via json load). FusionSegmenter is opt-in-only.

Retained despite weak n=6 lift ? No disable-because-weak commit exists. QUALITY_ITERATIONS.md:54-60 explicitly: "both stay default-OFF (promote-only-on-lift)... Negative/uncertain results are kept on purpose ? this is the honest ablation backbone." Producers + feature math + scorer columns all still present and re-measurable.

No special-casing (weighted columns) ? Confirmed: cues enter the learned decision only via `Variant.feature_names` ? `_feature_matrix` ? `model.predict_proba` (experiment.py:227-238). `LogisticRegression` (classifier.py:64) learns one weight per column (`self.w = np.zeros(d)`) with zero per-cue branching. No `if audio_hit_count > X` / `if players...` branch in any learned path.

The audit's own honestly-flagged caveats hold up but are NOT violations of the four claims:
- The two served gates (`min_crossings`/`min_players` in `_gate_mask`:200-210 and `_select_for_handoff`:146-148) ARE hard if/else thresholds ? but they are a deliberate, separate, default-OFF decision-gate (rally_gate's Gate A/B), not the path by which the cue enters the *learned* scorer, and they no-op at default thresholds.
- A minor data-hygiene footgun exists: `fusion_compare.load_videos`:47-48 and `fusion_audio.load_videos_with_audio`:66-67 index `c["person"][fn]`/`c["audio"][fn]` directly (KeyError if a future JSON lacks the dict), whereas the actual *scorer* path `_feature_column`:185-188 gracefully defaults a missing column to 0.0. This is a loader strictness inconsistency, not a "signal lost / special-cased" violation ? the loaders fail loudly rather than silently dropping a signal, and the learned scorer is back-compatible.

None of the targeted violations are present. Claim stands.

29. user (2026-06-14T06:37:16.655Z)

Structured output provided successfully

30. assistant (2026-06-14T06:37:26.989Z)

I re-read the actual code and could not refute the overall claim. **refuted=false.**

All three weak signals (person, audio, court-bounds) verifiably satisfy the four properties:

- **Present**: every producer and feature-math file is git-tracked and intact (`git log --diff-filter=D` over all cue files is empty).
- **Default-OFF**: `cue_flags={}`, `min_crossings/min_players=0`, `court_polygon=None`, `default_segmenter='yolo_hybrid'`, and shipped `config.json` has no fusion block (loads as `gemini`).
- **Retained despite weak n=6 ?F1**: no disable/delete commit exists; QUALITY_ITERATIONS.md:54-60 explicitly keeps them default-OFF "on purpose."
- **No special-casing**: cues enter the learned decision only as `feature_names` columns through `predict_proba`; the `LogisticRegression` learns one weight per column with zero per-cue branching.

The audit's flagged caveats hold but are not violations of the four claims:
- The served `min_crossings`/`min_players` gates are genuine hard if/else thresholds, but they're a deliberate, separately-documented, default-OFF Gate A/B ? not the path a cue takes into the learned scorer.
- One minor data-hygiene inconsistency: the eval loaders (`fusion_compare.py:47-48`, `fusion_audio.py:66-67`) index `c["person"]`/`c["audio"]` directly and would raise KeyError on a future JSON missing the dict, whereas the scorer's `_feature_column` gracefully defaults a missing column to 0.0. That's a fail-loud loader vs. back-compat scorer mismatch, not a silently-lost signal or a special-case.

The claim stands.